PYTHON ACTIVITY – 1

CASE STUDY PROGRAMS

Name: Shanmukha Sai

Roll Number: 26001002

Python programming case studies with problem descriptions, sample implementations, and policy notes.

Submitted as an academic activity

Contents

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

Case Study 1: University Scholarship Allocation

Objective: Checks aggregate marks, subject minimums, attendance, and family income. Eligible applicants are ranked by aggregate marks, then attendance, then lower family income.

Python Program

```
students = [
    {"name": "Asha", "marks": 88, "subjects": [80, 85, 90], "attendance": 92, "income": 180000},
    {"name": "Ravi", "marks": 91, "subjects": [75, 82, 88], "attendance": 89, "income": 220000},
    {"name": "Maya", "marks": 84, "subjects": [70, 78, 80], "attendance": 95, "income": 150000},
]

MIN_AGGREGATE = 80
MIN_SUBJECT = 60
MIN_ATTENDANCE = 75
MAX_INCOME = 250000

eligible = []
for s in students:
    if (s["marks"] >= MIN_AGGREGATE
        and min(s["subjects"]) >= MIN_SUBJECT
        and s["attendance"] >= MIN_ATTENDANCE
        and s["income"] <= MAX_INCOME):
        eligible.append(s)

eligible.sort(key=lambda s: (-s["marks"], -s["attendance"], s["income"], s["name"]))
print("Scholarship ranking")
for rank, s in enumerate(eligible, start=1):
    print(rank, s["name"], "Marks:", s["marks"],
          "Attendance:", s["attendance"], "Income:", s["income"])
```

Policy note: Sample output: Ravi ranks first, followed by Asha, based on the configured marks and tie-break rules.

Case Study 2: Progressive Electricity Billing

Objective: Calculates a household electricity bill using progressive consumption slabs, fixed charges, surcharge, and rebate. It compares monthly totals and flags a large increase.

Python Program

```
def energy_charge(units):
    slabs = [(100, 1.5), (100, 2.5), (200, 4.0), (float("inf"), 6.0)]
    charge, remaining = 0, units
    for limit, rate in slabs:
        used = min(remaining, limit)
        charge += used * rate
        remaining -= used
        if remaining <= 0:
            break
    return charge

def bill(units):
    fixed = 75
    subtotal = energy_charge(units) + fixed
    surcharge = subtotal * 0.05
    rebate = 25 if units < 100 else 0
    return round(subtotal + surcharge - rebate, 2)

monthly_units = {"Jan": 90, "Feb": 110, "Mar": 260, "Apr": 420}
monthly_bills = {month: bill(units) for month, units in monthly_units.items()}

for month, amount in monthly_bills.items():
    print(month, "units =", monthly_units[month], "bill = Rs.", amount)

months = list(monthly_bills)
for i in range(1, len(months)):
    old, new = monthly_bills[months[i-1]], monthly_bills[months[i]]
    increase = (new - old) / old * 100 if old else 0
    if increase > 40:
        print("Unusual increase:", months[i-1], "to", months[i],
              f"({increase:.1f}%)")
```

Policy note: Note: slab rates and the 40% alert threshold are example policy values and can be changed.

Case Study 3: Hospital Billing and Patient Classification

Objective: Adds consultation, diagnostic, and treatment costs, applies a concession when eligible, validates the bill, and classifies payment status using an example policy.

Python Program

```
def create_bill(consultation, diagnostics, treatment, concession_percent,
                amount_paid):
    charges = [consultation, diagnostics, treatment]
    if any(x < 0 for x in charges):
        raise ValueError("Charges cannot be negative")
    gross = sum(charges)
    if not 0 <= concession_percent <= 100:
        raise ValueError("Invalid concession percentage")
    concession = gross * concession_percent / 100
    net = round(gross - concession, 2)
    if amount_paid < 0:
        raise ValueError("Payment cannot be negative")
    balance = round(net - amount_paid, 2)

    if balance <= 0:
        status = "Paid"
    elif amount_paid > 0:
        status = "Partially paid"
    else:
        status = "Unpaid"

    return gross, concession, net, max(balance, 0), status

gross, discount, net, balance, status = create_bill(
    consultation=500, diagnostics=1200, treatment=3500,
    concession_percent=10, amount_paid=3000
)
print("Gross: Rs.", gross)
print("Concession: Rs.", discount)
print("Net bill: Rs.", net)
print("Balance due: Rs.", balance)
print("Patient payment status:", status)
```

Policy note: Concessions and patient labels are illustrative; apply the actual policy supplied by the institution.

Case Study 4: Employee Payroll and Deduction Analysis

Objective: Computes gross salary from basic pay, allowances, overtime, and incentives, then applies progressive deductions and prints a salary statement.

Python Program

```
def progressive_tax(amount):
    tax = 0
    brackets = [(20000, 0.00), (30000, 0.05), (50000, 0.10),
                (float("inf"), 0.20)]
    previous = 0
    for limit, rate in brackets:
        taxable = min(max(amount - previous, 0), limit - previous)
        tax += taxable * rate
        previous = limit
        if amount <= limit:
            break
    return tax

basic = 30000
allowances = 8000
overtime_hours = 10
overtime_rate = 250
incentives = 3000
provident_fund_rate = 0.08

overtime = overtime_hours * overtime_rate
gross = basic + allowances + overtime + incentives
pf = basic * provident_fund_rate
tax = progressive_tax(gross)
other_deduction = 500
total_deductions = pf + tax + other_deduction
net = gross - total_deductions

print("SALARY STATEMENT")
print("-" * 30)
print(f"Basic pay:          Rs. {basic:,.2f}")
print(f"Allowances:         Rs. {allowances:,.2f}")
print(f"Overtime:           Rs. {overtime:,.2f}")
print(f"Incentives:         Rs. {incentives:,.2f}")
print(f"Gross salary:       Rs. {gross:,.2f}")
print(f"PF deduction:       Rs. {pf:,.2f}")
print(f"Tax deduction:      Rs. {tax:,.2f}")
print(f"Other deductions:   Rs. {other_deduction:,.2f}")
print(f"Net salary:         Rs. {net:,.2f}")
```

Policy note: Tax brackets and deduction rates are examples for demonstrating progressive calculations.

Case Study 5: Multi-Plan Telecommunications Cost Optimization

Objective: Estimates total monthly cost for mobile plans based on included data, calls, validity, and budget. It selects the least expensive plan that meets the usage requirements.

Python Program

```
usage_data_gb = 18
usage_minutes = 500
days_needed = 28
budget = 600

plans = [
    {"name": "Basic", "price": 299, "data": 10, "minutes": 300,
     "validity": 28, "extra_data": 20, "extra_minute": 0.5},
    {"name": "Smart", "price": 449, "data": 25, "minutes": 700,
     "validity": 28, "extra_data": 15, "extra_minute": 0.4},
    {"name": "Max", "price": 599, "data": 50, "minutes": 1000,
     "validity": 56, "extra_data": 10, "extra_minute": 0.3},
]

eligible = []
for p in plans:
    extra_data = max(0, usage_data_gb - p["data"])
    extra_minutes = max(0, usage_minutes - p["minutes"])
    total = p["price"] + extra_data * p["extra_data"] + extra_minutes * p["extra_minute"]
    meets_usage = p["validity"] >= days_needed
    if total <= budget and meets_usage:
        eligible.append((total, p["name"], extra_data, extra_minutes))

if eligible:
    eligible.sort()
    total, name, extra_data, extra_minutes = eligible[0]
    print("Most economical eligible plan:", name)
    print("Estimated cost: Rs.", round(total, 2))
    print("Excess data (GB):", extra_data)
    print("Excess minutes:", extra_minutes)
else:
    print("No plan meets the usage, validity, and budget constraints.")
```

Policy note: Excess-charge units and rates are illustrative; actual plans may bill data in smaller units.

Case Study 6: Railway Fare and Reservation Calculator

Objective: Calculates passenger fares using distance-based base rates, travel-class multipliers, age concessions, reservation fees, and cancellation penalties.

Python Program

```
def base_fare(distance_km):
    if distance_km <= 100:
        return distance_km * 1.2
    elif distance_km <= 500:
        return 100 * 1.2 + (distance_km - 100) * 1.0
    return 100 * 1.2 + 400 * 1.0 + (distance_km - 500) * 0.8

def passenger_fare(distance, travel_class, age):
    class_multiplier = {"Sleeper": 1.0, "AC3": 1.8, "AC2": 2.5}
    if travel_class not in class_multiplier:
        raise ValueError("Unknown travel class")
    fare = base_fare(distance) * class_multiplier[travel_class]
    if age >= 60:
        fare *= 0.75
    elif age < 12:
        fare *= 0.50
    return round(fare + 30, 2)  # reservation fee

passengers = [
    {"name": "Arun", "age": 35, "class": "Sleeper"},
    {"name": "Meena", "age": 65, "class": "AC3"},
    {"name": "Kiran", "age": 10, "class": "Sleeper"},
]
distance = 320
for p in passengers:
    fare = passenger_fare(distance, p["class"], p["age"])
    print(p["name"], p["class"], "fare: Rs.", fare)

cancellation_percent = 25
fare_to_cancel = passenger_fare(distance, "Sleeper", 35)
penalty = fare_to_cancel * cancellation_percent / 100
print("Cancellation penalty: Rs.", round(penalty, 2))
```

Policy note: Age concessions, class multipliers, reservation fees, and cancellation rates must be replaced with the supplied railway policy.

Case Study 7: Retail Billing with Multiple Discount Rules

Objective: Creates an itemized bill with quantity, category-specific tax, threshold discounts, and a maximum concession cap.

Python Program

```
products = [
    {"name": "Notebook", "category": "stationery", "price": 60, "qty": 3},
    {"name": "Headphones", "category": "electronics", "price": 800, "qty": 1},
    {"name": "Bottle", "category": "home", "price": 150, "qty": 2},
]
tax_rates = {"stationery": 0.05, "electronics": 0.18, "home": 0.12}
discount_rate = 0.10
discount_threshold = 1000
max_discount = 250

subtotal = 0
tax_total = 0
print("ITEMIZED INVOICE")
for p in products:
    amount = p["price"] * p["qty"]
    tax = amount * tax_rates[p["category"]]
    subtotal += amount
    tax_total += tax
    print(f'{p["name"]:<15} {p["qty"]:>3} x Rs.{p["price"]:>7.2f} '
          f'Amount Rs.{amount:>8.2f} Tax Rs.{tax:>7.2f}')

discount = min(subtotal * discount_rate, max_discount) if subtotal >= discount_threshold else 0
grand_total = subtotal + tax_total - discount
print(f"Subtotal: Rs. {subtotal:.2f}")
print(f"Tax total: Rs. {tax_total:.2f}")
print(f"Discount: Rs. {discount:.2f}")
print(f"Grand total: Rs. {grand_total:.2f}")
```

Policy note: Tax rates and discount conditions are examples. Confirm whether discounts apply before or after tax for the required policy.

Case Study 8: Manufacturing Cost and Profitability Analysis

Objective: Calculates material, labour, wastage, and machine costs, then evaluates the profit margin against a target.

Python Program

```
units = 500
material_cost_per_unit = 80
labour_cost_per_unit = 25
wastage_percent = 5
machine_cost = 6000
selling_price_per_unit = 140
target_profit = 10000

material_base = units * material_cost_per_unit
wastage_cost = material_base * wastage_percent / 100
labour_cost = units * labour_cost_per_unit
total_cost = material_base + wastage_cost + labour_cost + machine_cost
revenue = units * selling_price_per_unit
profit = revenue - total_cost
margin = (profit / revenue * 100) if revenue else 0

print("Material cost: Rs.", material_base)
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Labour cost: Rs.", labour_cost)
print("Machine cost: Rs.", machine_cost)
print("Total production cost: Rs.", round(total_cost, 2))
print("Sales revenue: Rs.", revenue)
print("Profit: Rs.", round(profit, 2))
print(f"Profit margin: {margin:.2f}%")
print("Target status:", "Achieved" if profit >= target_profit else "Not achieved")
```

Policy note: Assumes wastage is calculated as a percentage of base material cost.

Case Study 9: Multi-Currency Travel Budget Analysis

Objective: Converts expenses in multiple currencies using supplied exchange rates and conversion fees, compares total spending with a budget, and finds the largest category.

Python Program

```
budget_inr = 80000
exchange_rates = {"USD": 83.0, "EUR": 90.0, "GBP": 105.0}
conversion_fee_rate = 0.02
expenses = [
    {"category": "Hotel", "amount": 300, "currency": "USD"},
    {"category": "Food", "amount": 150, "currency": "EUR"},
    {"category": "Transport", "amount": 100, "currency": "USD"},
    {"category": "Shopping", "amount": 120, "currency": "GBP"},
]

totals = {}
overall = 0
for item in expenses:
    currency = item["currency"]
    if currency not in exchange_rates:
        print("Missing exchange rate for", currency)
        continue
    converted = item["amount"] * exchange_rates[currency]
    fee = converted * conversion_fee_rate
    total = converted + fee
    totals[item["category"]] = total
    overall += total
    print(item["category"], "=", f"Rs. {total:.2f}")

largest = max(totals, key=totals.get) if totals else "N/A"
print(f"Total travel spending: Rs. {overall:.2f}")
print(f"Budget: Rs. {budget_inr:.2f}")
print("Budget status:", "Within budget" if overall <= budget_inr else "Over budget")
print("Largest expense category:", largest)
```

Policy note: Exchange rates are sample inputs only; use the rates supplied for the activity.

Case Study 10: Campus Event Financial Planning

Objective: Adds event expenses, calculates the participant contribution after sponsorship, estimates a sponsorship shortfall, and finds the minimum registrations needed to break even.

Python Program

```
venue = 12000
equipment = 8000
catering_per_person = 180
transport_per_person = 40
certificate_per_person = 20
sponsorship = 15000
registration_fee = 300
expected_participants = 100

fixed_cost = venue + equipment
variable_per_person = catering_per_person + transport_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * expected_participants
shortfall_after_sponsorship = max(0, total_cost - sponsorship)
required_contribution = shortfall_after_sponsorship / expected_participants

# Break-even registrations when each registration contributes registration_fee
contribution_per_registration = registration_fee - variable_per_person
if contribution_per_registration <= 0:
    break_even = None
else:
    break_even = max(0, int((fixed_cost - sponsorship) / contribution_per_registration))
    if break_even * contribution_per_registration < fixed_cost - sponsorship:
        break_even += 1

print("Fixed costs: Rs.", fixed_cost)
print("Variable cost per participant: Rs.", variable_per_person)
print("Total event cost: Rs.", total_cost)
print("Sponsorship: Rs.", sponsorship)
print("Funding shortfall: Rs.", shortfall_after_sponsorship)
print("Required contribution per expected participant: Rs.", round(required_contribution, 2))
print("Break-even registrations:", break_even if break_even is not None else
      "Not possible at this registration fee")
```

Policy note: Break-even assumes sponsorship is available toward fixed costs and each registration pays the stated fee.